OOPs Introduction in JAVA

1. What is OOPs?

OOPs (Object-Oriented Programming System) is a programming paradigm based on the concept of objects.

In Java, everything revolves around objects and classes.

Simple Definition:

OOPs is a way of writing programs using objects and classes instead of just functions.

2. Why OOPs?

Before OOPs → Procedural Programming (C language style)

Problems:

- Code duplication
- Difficult to manage large projects
- No proper security

OOPs solves:

- Reusability
- Security
- Easy maintenance
- Real-world modeling

3. Real-Life Example of OOPs

Think of a Car

- Class → Car
- Object → BMW, Audi, Tesla

Properties:

- color
- speed

Behavior:

- start()
- stop()

4. Basic Concepts of OOPs

There are 4 main pillars:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

5. Class in Java

Definition:

A class is a blueprint or template for creating objects.

Syntax:

```
class Student {
   int id;
   String name;
}
```

6. Object in Java

- **Definition:**

An object is an instance of a class.

- **Syntax:**

```
Student s1 = new Student();
```

7. Full Example (Class + Object)

```
class Student {
   int id;
   String name;

   void display() {
      System.out.println(id + " " + name);
   }
}

public class Main {
   public static void main(String[] args) {
      Student s1 = new Student();

      s1.id = 1;
      s1.name = "Omii";

      s1.display();
   }
}
```

8. Constructor in Java

Definition:

A constructor is a special method used to initialize objects.

Example:

```
class Student {
   int id;
   String name;

   Student(int i, String n) {
      id = i;
      name = n;
   }
```

```
  void display() {
    System.out.println(id + " " + name);
  }
}
```

9. Types of Constructors

1. Default Constructor
2. Parameterized Constructor

10. Encapsulation

Definition:

Wrapping data and methods into a single unit.

Achieved using:

- Private variables
- Public getter/setter methods

Example:

```
class Student {
  private int id;

  public void setId(int id) {
    this.id = id;
  }

  public int getId() {
    return id;
  }
}
```

11. Inheritance

Definition:

One class acquires properties of another class.

Syntax:

```
class Animal {
  void eat() {
    System.out.println("Eating...");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking...");
  }
}
```

12. Types of Inheritance

1. Single
2. Multilevel
3. Hierarchical

(Java does NOT support multiple inheritance with classes)

13. Polymorphism

Definition:

One method, multiple forms.

Types:

1. Compile-time (Method Overloading)

2. Runtime (Method Overriding)

Method Overloading Example:

```
class MathUtil {
  int add(int a, int b) {
    return a + b;
  }

  int add(int a, int b, int c) {
    return a + b + c;
  }
}
```

Method Overriding Example:

```
class Animal {
  void sound() {
    System.out.println("Animal sound");
  }
}

class Dog extends Animal {
  void sound() {
    System.out.println("Dog barks");
  }
}
```

14. Abstraction

Definition:

Hiding implementation details and showing only functionality.

Achieved using:

- Abstract classes

- Interfaces

Abstract Class Example:

```
abstract class Animal {
  abstract void sound();
}

class Dog extends Animal {
  void sound() {
    System.out.println("Bark");
  }
}
```

15. Interface in Java

```
interface Animal {
  void sound();
}

class Dog implements Animal {
  public void sound() {
    System.out.println("Bark");
  }
}
```

16. Difference: Class vs Object

Class	**Object**
Blueprint	Instance
Logical	Physical
No memory	Takes memory

17. Difference: Abstraction vs Encapsulation

Abstraction	Encapsulation
Hides implementation	Hides data
Abstract class/interface	Getter/Setter

18. Real-World Project Example

Employee System

```
class Employee {
   int id;
   String name;
   double salary;

   void display() {
      System.out.println(id + " " + name + " " + salary);
   }
}
```

19. Advantages of OOPs

✓ Code Reusability
✓ Security
✓ Easy Maintenance
✓ Modularity
✓ Flexibility

20. Disadvantages of OOPs

Complex for beginners
More memory usage
Slower than procedural

- Java supports OOPs fully except multiple inheritance (with classes)
- Everything in Java is object-based except primitives
- Constructor is not a method
- Method overriding → runtime polymorphism

21. Conclusion

OOPs is the heart of Java programming.
Without understanding OOPs, you cannot become a strong Java developer.